



Міністерство освіти і науки України
Національний технічний університет України
“Київський політехнічний інститут імені Ігоря Сікорського”
Факультет інформатики та обчислювальної техніки
Кафедра інформаційних систем та технологій

Технології розроблення програмного забезпечення
Лабораторна робота №7
на тему «ШАБЛОНИ «MEDIATOR», «FACADE», «BRIDGE»,
«TEMPLATE METHOD».»

Виконала:
Студентка групи ІА-22
Патрик Вікторія

Перевірив:
Мягкий Михайло Юрійович

Київ 2024

Зміст

Тема	3
Короткі теоретичні відомості.....	3
Реалізація не менше 3-х класів відповідно до обраної теми	10
Реалізація шаблону за обраною темою	12
1. MacroProcessor.....	12
2. FormattingMacroProcessor	13
3. SnippetMacroProcessor	14
4. MacroController	15
5. MacroService.....	16
Посилання на код на GitHub	16
Діаграма класів для патерну «Template method»	17
Переваги застосування патерну «Template method».....	18

ШАБЛОНИ «MEDIATOR», «FACADE», «BRIDGE», «TEMPLATE METHOD»

Мета: ознайомитися з шаблонами проектування «MEDIATOR», «FACADE», «BRIDGE», «TEMPLATE METHOD» та набуті практичних навичок їх застосування. Реалізувати частину функціоналу програми за допомогою одного з розглянутих шаблонів для досягнення конкретних функціональних можливостей та забезпечення ефективної взаємодії між класами.

Тема:

..3 Текстовий редактор (strategy, command, observer, template method, flyweight, SOA)

Текстовий редактор повинен вміти розпізнавати текстові файли в будь-якій кодуванні, мати розширені функції редагування: макроси, сніппети, підказки, закладки, перехід на рядок / сторінку, підсвічування синтаксису (для однієї мови програмування або розмітки на розсуд студента).

Хід роботи:

Короткі теоретичні відомості:

Принципи проектування:

1. Принцип Don't Repeat Yourself (DRY) свідчить, що в початкових кодах програмного продукту не повинно бути повторень. Ці повторення можуть спостерігатися на різних рівнях - від повторень наборів рядків коду (абсолютно аналогічні рядки в різних місцях проекту) до різної реалізації функціональності, що повторюється. Повторень уникати слід з кількох причин: передусім, код без повторень значно менше і значно легше читається; по-друге, при знаходженні помилки в коді, що має повторення, є високий шанс не знати/забути виправити цю ж помилку у іншому місці; по-третє, при створенні ідентичного коду ви не лише копіюєте функціональність, але і помилки; по-четверте, при необхідності додавання нових можливостей скрізь де є повторення необхідно внести відповідні коректури. Повторення легко виправити застосувавши відповідну техніку рефакторинга - винесення метода/інтерфейса/класа та ін.

2. Принцип Keep it simple, Stupid! (KISS) пропонує уникати зайвого ускладнення компонентів системи. Вважається, що система, яка складається з безлічі маленьких простих частин, працює значно надійніше, ніж одна велика

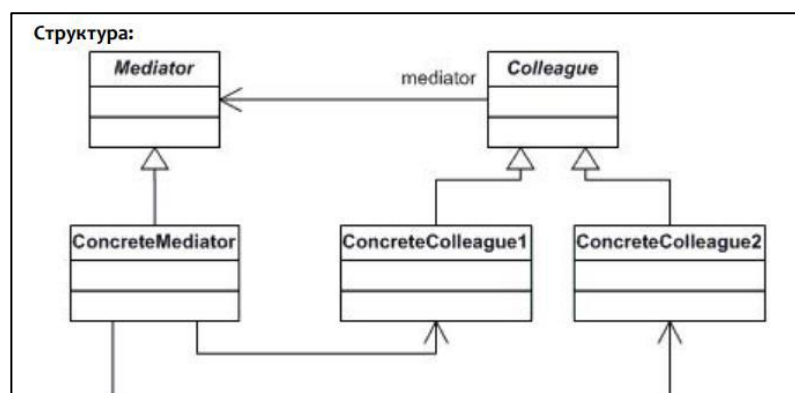
складна система. Даною філософією слідує співтовариство Unix, де кожна програма виконує рівно одну функцію, але при цьому виконує її абсолютно добре (Do one thing right). Це також сприяє задоволенню відомого афоризму «The system obviously has no errors rather than system has no obvious errors» (очевидно, що в системі немає помилок; в системі немає очевидних помилок»). Використання простих конструкцій спрощує читаємість і сприйняття вихідних кодів.

3. Принцип You only load it once! (YOLO) вказує на необхідність підвантаження ініціалізаційних і конфігураційних змінних один раз при запуску програми, щоб уникнути проблем зі швидкістю (повторні зчитування даних з вінчестера).

4. Принцип Парето застосовується до безлічі різних подій і визначає відсоткове співвідношення 80-20. Декілька прикладів застосування принципу Парето: - 80% навантаження на сервер створює 20% додатка; - 80% всього вихідного коду програми пишеться за 20% часу; - 80% помилок програмного забезпечення можна усунути закривши лише 20% багів. Таким чином, необхідно пам'ятати про те, що для вирішення більшості проблем необхідно докласти лише малу частину зусиль. Однак для доведення додатку до стану досконалості може коштувати величезну суму (80%).

5. Принцип You ain't gonna need it пропонує відмовитися від деяких рішень в силу того, що вони просто не знадобляться. Досить відомий факт, що програмісти люблять узагальнювати та надавати універсальні рішення на всі випадки життя. Дуже часто досить простого і працюючого рішення замість універсального, оскільки інші випадки навряд чи будуть розглядатися в подальшому. Є необхідний набір функціональності, який необхідно реалізувати; додавання зайвої гнучкості не тільки захарастить код, але і зменшить хід процесу розробки.

Шаблон «Mediator» (Посередник)



Призначення: Шаблон «Mediator» забезпечує взаємодію між об'єктами через окремий об'єкт-посередник, замість прямого зв'язку між компонентами. Це дозволяє уникнути складних залежностей і робить код більш гнучким і повторно використовуваним.

Проблема: При взаємодії елементів діалогу, таких як текстові поля, кнопки та чекбокси, складна логіка може ускладнити повторне використання компонентів у різних контекстах.

Рішення: Посередник координує взаємодії між елементами, знижуючи їх залежність один від одного і дозволяючи використовувати компоненти в різних контекстах.

Приклад з життя: Пілоти літаків не спілкуються безпосередньо між собою, а через диспетчера, який координує їх взаємодію.

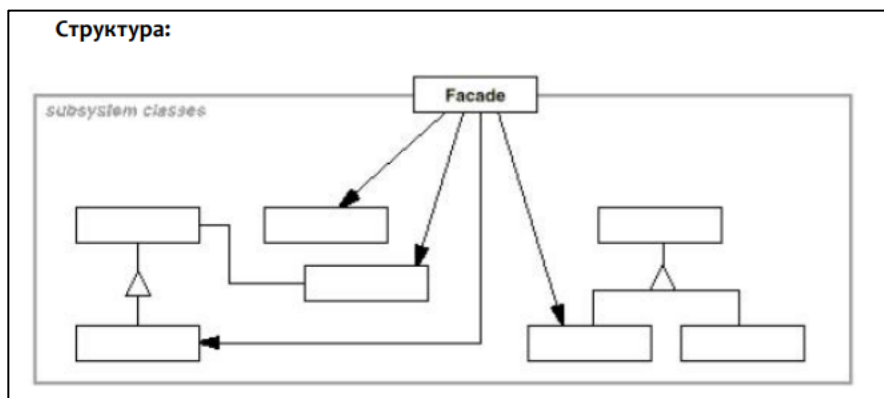
Переваги:

- Знижує залежності між компонентами.
- Спрощує взаємодію та централізує управління.

Недоліки:

- Посередник може стати занадто великим і складним.

Шаблон «Facade» (Фасад)



Призначення: Шаблон «Facade» створює єдиний, уніфікований спосіб доступу до складної підсистеми, приховуючи її внутрішні деталі. Це дозволяє спростити взаємодію з підсистемою, надаючи користувачу простий інтерфейс, при цьому приховуючи складність реалізації.

Проблема: При роботі з великою кількістю об'єктів складної бібліотеки або фреймворка бізнес-логіка може переплітатися з деталями реалізації сторонніх класів, що ускладнює підтримку коду.

Рішення: Фасад надає простий інтерфейс для взаємодії з підсистемою, зберігаючи тільки необхідну функціональність для клієнта, приховуючи інші складності.

Приклад з життя: Якщо ви телефонуєте в магазин для оформлення замовлення, співробітник служби підтримки є фасадом, який спрощує доступ до різних служб (замовлення, оплата, доставка).

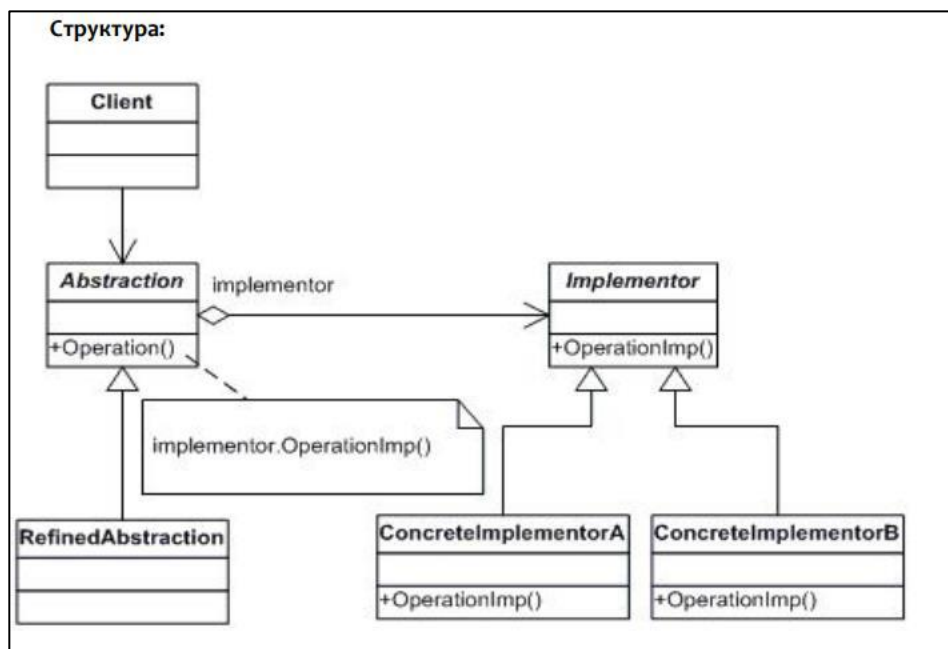
Переваги:

- Ізолює клієнта від складної підсистеми.
- Спрощує використання складних бібліотек або фреймворків.

Недоліки:

- Фасад може стати занадто великим об'єктом, що взаємодіє з усіма компонентами програми.

Шаблон «Bridge» (Міст)



Призначення: Шаблон «Bridge» використовується для поділу абстракції та її реалізації. Це особливо корисно, коли існують різні абстракції, і їх можна реалізувати різними способами. Він дозволяє відокремити абстракцію від її реалізації та додавати нові абстракції незалежно від реалізацій.

Проблема: Якщо ви намагаєтесь комбінувати кілька властивостей (наприклад, колір і форма), для кожної нової комбінації доведеться створювати нові класи, що призводить до різкого збільшення кількості класів та ускладнення підтримки. Наприклад, у разі додавання нових фігур або кольорів потрібно

створювати нові підкласи для кожної можливості, що швидко стає неефективним.

Рішення: Шаблон «Bridge» вирішує цю проблему, замінюючи спадкування агрегацією або композицією. Винятковим чином, можна створити окрему ієрархію для кожної «площини» (наприклад, колір), а потім використовувати ці об'єкти в класі абстракції (наприклад, фігури). Таким чином, при додаванні нових кольорів не потрібно змінювати класи фігур і навпаки.

Приклад з життя: Уявіть два міста, розділені річкою. Для з'єднання між ними будують міст. Кожне місто розвивається за своїми законами, але міст залишається сталим, і міста підлаштовуються під міст. Міст є «мостом» між містами, дозволяючи їм взаємодіяти, не змінюючи їхню власну сутність.

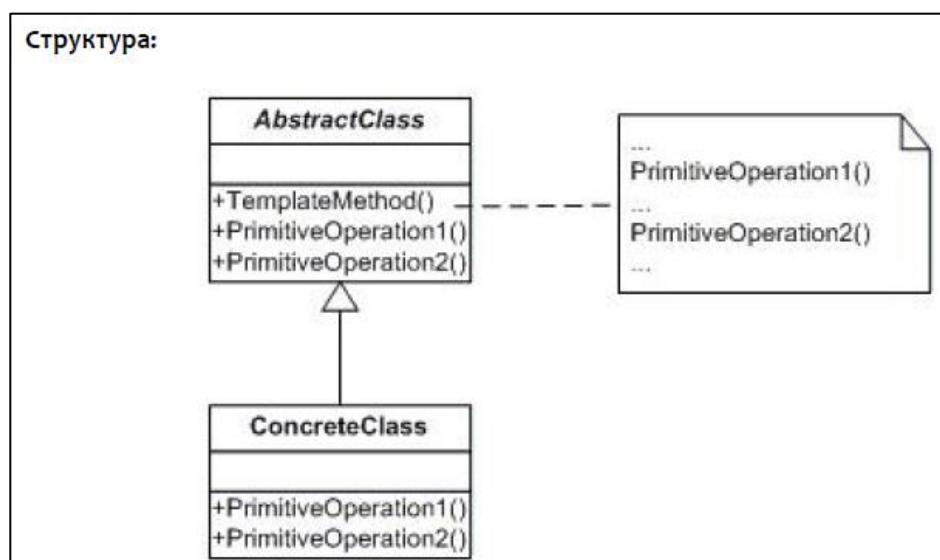
Переваги:

- Дозволяє створювати платформи-незалежні програми.
- Приховує зайві або небезпечні деталі реалізації від клієнтського коду.
- Реалізує принцип відкритості/закритості (легкість додавання нових функціональностей без зміни існуючого коду).

Недоліки:

- Ускладнює код через додавання нових класів і шарів абстракції.

Шаблон «Template Method»



Призначення патерну: Шаблон «Template Method» (шаблонний метод) дозволяє реалізувати покроковий алгоритм в абстрактному класі, але залишити специфіку реалізації для підкласів. Він дозволяє визначити загальний шаблон для алгоритму, а конкретні кроки цього алгоритму

залишати на розсуд підкласів. Це зручно, коли потрібно забезпечити однакову структуру алгоритмів, але з можливістю змінювати конкретні кроки в залежності від потреб.

Приклад використання: Уявіть собі систему, яка формує веб-сторінки. Алгоритм формування сторінки включає кілька кроків:

- Формування заголовків,
- Формування контенту,
- Формування файлів, що додаються,
- Формування нижньої частини сторінки.

Ці кроки будуть однаковими для різних типів сторінок (наприклад, HTML, PHP, ASP.NET), але специфіка реалізації деяких кроків (наприклад, додавання вмісту) може варіюватися в залежності від типу сторінки.

Ось приклад на мові C# використання даного шаблону:

```
public class PageFormer
{
    void FormHeaders() { ... }
    void FormFooters() { ... }
    void FormAddedFiles() { ... }
    abstract void FormContent();
    void FormPage()
    {
        FormHeaders();
        FormContent();
        FormFooters();
        FormAddedFiles();
    }
}

public class AspNetCompiler : PageFormer
{
    override void FormContent() { ... }
}
```

Проблема: При розробці програми для дата-майнінгу документів, де потрібно обробляти різні формати (наприклад, PDF, DOC, CSV), спочатку обробляються лише DOC-файли. Пізніше додаються інші формати, і з'являється багато повторень в коді для кожного типу файлів, оскільки процес вилучення даних є схожим у всіх випадках.

Рішення: Шаблон «Template Method» дозволяє описати загальний алгоритм обробки документів, при цьому залишити деталі реалізації (наприклад, як саме зчитувати DOC, CSV або PDF) для підкласів. Таким чином, спільний код з вилучення даних можна реалізувати в базовому класі, а конкретні кроки, які відрізняються для кожного типу файлів, залишити для перевизначення в підкласах.

Приклад з життя: Будівельники можуть використовувати шаблонний метод при будівництві типових будинків. Зазначено, що будівництво включає стандартні етапи (фундамент, стіни, дах), але на кожному етапі можливі невеликі варіації, наприклад, типи матеріалів або техніка виконання. Саме ці варіації підклас будівельників може реалізувати, при цьому дотримуючись загального плану.

Переваги та недоліки:

Переваги:

- Полегшує повторне використання коду, оскільки спільні частини алгоритму реалізуються в базовому класі.
- Спрощує підтримку, оскільки можна змінювати конкретні кроки, не торкаючись загальної структури алгоритму.

Недоліки:

- Жорстко обмежує можливість зміни структури алгоритму через базовий клас.
- Може порушити принцип підстановки Барбари Лісков, якщо підклас змінює поведінку одного з кроків таким чином, що алгоритм більше не працює коректно.
- При великій кількості кроків шаблонний метод може стати занадто складним для підтримки.

Цей патерн дуже корисний, коли є необхідність забезпечити повторне використання загальних кроків алгоритму, при цьому дозволяючи підкласам змінювати або доповнювати лише частини алгоритму.

Реалізація не менше 3-х класів відповідно до обраної теми:

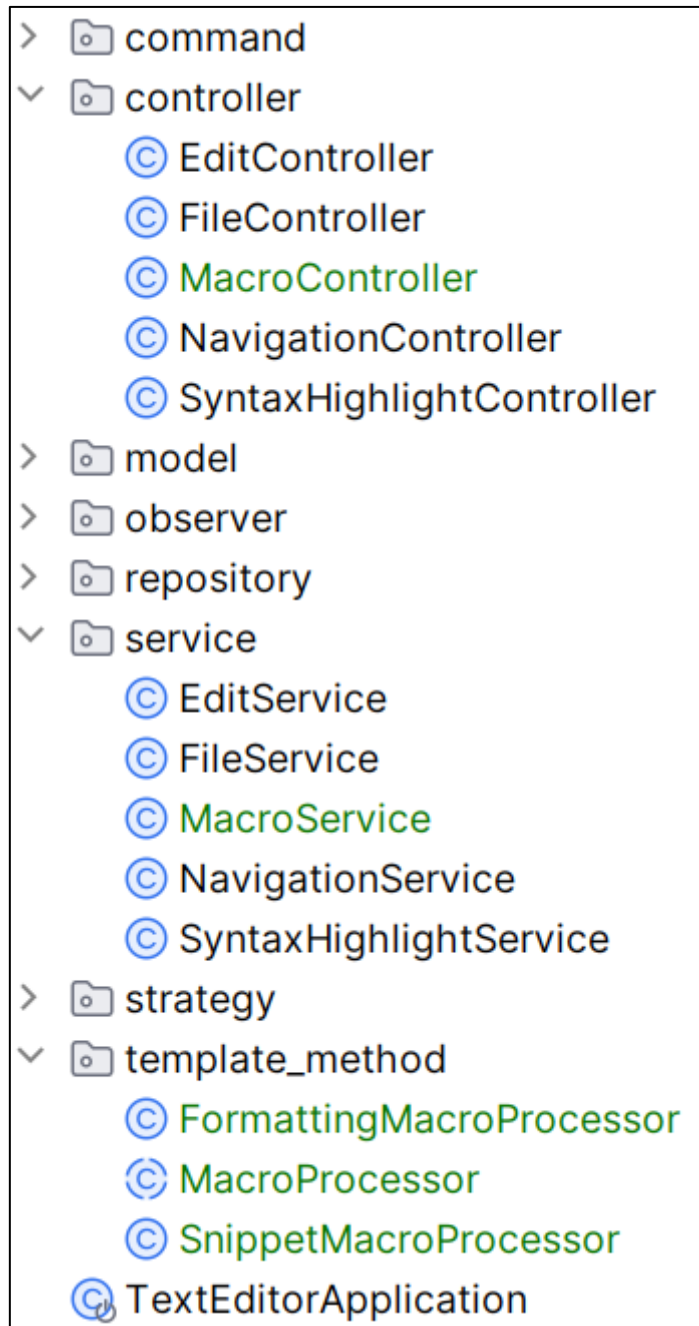


Рисунок 1. Структура проекту з використанням патерну «Template method».

У ході лабораторної роботи була реалізована система текстового редактора, основним завданням якої було використання патерну **Template Method** для організації роботи з макросами та підвищення гнучкості функціоналу текстового редактора. Структура проекту наведена на Рисунку 1. У результаті виконання роботи були реалізовані наступні кроки:

1. Розроблено структуру проекту:

- Проект поділено на логічні пакети: controller, model, repository, service, strategy та template_method, що забезпечує зручність масштабування та підтримки коду.

2. Реалізовано патерн " **Template Method** ":

- Створено абстрактний клас `MacroProcessor`, який визначає алгоритм обробки макросів. Алгоритм включає:
 - Перевірку валідності макросу.
 - Витяг команди з тексту макросу.
 - Виконання команди.
 - Збереження результатів.
- Розроблено конкретні реалізації:
 - `FormattingMacroProcessor` — реалізація для обробки макросів форматування тексту.
 - `SnippetMacroProcessor` — реалізація для обробки макросів вставки сніппетів.

3. Реалізовано сервісний рівень:

- Створено клас `MacroService`, який забезпечує логіку обробки макросів із можливістю розширення функціоналу:
 - Метод `processFormattingMacro` відповідає за обробку макросів форматування.
 - Метод `processSnippetMacro` реалізує обробку макросів для вставки сніппетів.
- Всі методи сервісу працюють із відповідними реалізаціями патерну `Template Method`, що забезпечує гнучкість і легкість у розширенні функціоналу.

4. Створено контролери для роботи з функціоналом:

- `MacroController` — обробляє запити, пов'язані з макросами (форматування тексту, вставка сніппетів).

5. Перевірено функціонал системи:

- Тестування API підтвердило коректність виконання макросів (вставка сніппетів та форматування тексту).
- Система спроектована так, щоб її можна було легко розширити, додаючи нові типи макросів без змін у наявному коді.

Цей підхід забезпечує зручність у розширенні функціоналу текстового редактора, дозволяючи легко додавати нові можливості, такі як нові види макросів. Використання патерну **Template Method** у поєднанні з сервісним рівнем забезпечує єдину структуру для різних алгоритмів обробки макросів, спрощує організацію коду та підвищує зручність для розробників.

Реалізація шаблону за обраною темою:

1. MacroProcessor

```
1 package com.example.text_editor.template_method;
2
3 5 usages 2 inheritors new *
4 @ public abstract class MacroProcessor {
5
6     2 usages new *
7     public final void processMacro(String macroText) {
8         if (isValidMacro(macroText)) {
9             String command = extractCommand(macroText);
10            executeCommand(command);
11            saveResults();
12        } else {
13            System.out.println("Invalid macro: " + macroText);
14        }
15    }
16
17 1 usage 2 implementations new *
18 protected abstract boolean isValidMacro(String macroText);
19
20 1 usage 2 implementations new *
21 protected abstract String extractCommand(String macroText);
22
23 1 usage 2 implementations new *
24 protected abstract void executeCommand(String command);
25
26 1 usage new *
27 protected void saveResults() {
28     System.out.println("Saving results after executing macro...");
29 }
30 }
```

Рисунок 2. Абстрактний клас MacroProcessor.

Цей абстрактний клас, представлений на Рисунку 2, визначає шаблонний метод `processMacro(String macroText)`, який керує послідовністю виконання макросів. Алгоритм обробки включає:

- Перевірку валідності макросу через `isValidMacro(String macroText)`.
- Витяг команди макросу через `extractCommand(String macroText)`.
- Виконання команди через `executeCommand(String command)`.
- Збереження результатів виконання через метод `saveResults()`.

Використання класу `MacroProcessor` забезпечує спільний підхід до виконання макросів і дозволяє реалізувати різні види макросів через конкретні підкласи.

2. FormattingMacroProcessor

```
1 package com.example.text_editor.template_method;
2 2 usages new *
3
4 public class FormattingMacroProcessor extends MacroProcessor {
5
6     1 usage new *
7     @Override
8     protected boolean isValidMacro(String macroText) {
9         return macroText.startsWith("format:");
10    }
11
12     1 usage new *
13     @Override
14     protected String extractCommand(String macroText) {
15         return macroText.replace(target: "format:", replacement: "");
16    }
17
18     1 usage new *
19     @Override
20     protected void executeCommand(String command) {
21         System.out.println("Executing formatting command: " + command);
22    }
23 }
```

Рисунок 3. Клас FormattingMacroProcessor.

Цей клас, показаний на Рисунок 3, розширює MacroProcessor і реалізує макроси для форматування тексту. Використовуючи методи:

- isValidMacro(String macroText) перевіряє, чи починається текст із ключового слова format:.
- extractCommand(String macroText) видаляє префікс format: із макросу.
- executeCommand(String command) виконує форматування тексту відповідно до команди.

FormattingMacroProcessor надає функціональність для виконання макросів форматування, таких як "перетворення тексту у верхній регістр".

3. SnippetMacroProcessor

```
1 package com.example.text_editor.template_method;
2 usages new *
3
4 public class SnippetMacroProcessor extends MacroProcessor {
5
6     1 usage new *
7     @Override
8     protected boolean isValidMacro(String macroText) {
9         return macroText.startsWith("snippet:");
10    }
11
12    1 usage new *
13    @Override
14    protected String extractCommand(String macroText) {
15        return macroText.replace(target: "snippet:", replacement: "");
16    }
17
18    1 usage new *
19    @Override
20    protected void executeCommand(String command) {
21        System.out.println("Inserting snippet: " + getSnippet(command));
22    }
23
24    1 usage new *
25    @ private String getSnippet(String key) {
26        return switch (key) {
27            case "hello" -> "Hello, world!";
28            case "date" -> "Today's date: 2024-12-11";
29            default -> "[Snippet not found]";
30        };
31    }
32 }
```

Рисунок 4. Клас SnippetMacroProcessor.

Цей клас, показаний на Рисунку 4, також розширює MacroProcessor і обробляє макроси для вставки шніпетів. Він реалізує:

- isValidMacro(String macroText) перевіряє, чи починається текст із ключового слова snippet:.
- extractCommand(String macroText) видаляє префікс snippet: із макросу.
- executeCommand(String command) вставляє відповідний шніпет.

SnippetMacroProcessor використовує бібліотеку шніпетів і дозволяє вставляти готові текстові шаблони, такі як "Hello, world!" чи поточну дату.

4. MacroController

```
1 package com.example.text_editor.controller;
2
3 import com.example.text_editor.service.MacroService;
4 import org.springframework.web.bind.annotation.*;
5
6 import java.util.Map;
7
8 new *
9 @RestController
10 @RequestMapping("/macros")
11 public class MacroController {
12
13     3 usages
14     private final MacroService macroService;
15
16     new *
17     public MacroController(MacroService macroService) {
18         this.macroService = macroService;
19     }
20
21     new *
22     @PostMapping("/process/formatting")
23     public void processFormattingMacro(@RequestBody Map<String, String> request) {
24         String macroText = request.get("macroType");
25         if (macroText != null) {
26             macroService.processFormattingMacro(macroText);
27         } else {
28             System.out.println("Invalid request: macroType is missing");
29         }
30     }
31 }
```

Рисунок 5.1. Клас MacroController.

```
28 new *
29 @PostMapping("/process/snippet")
30 public void processSnippetMacro(@RequestBody Map<String, String> request) {
31     String macroText = request.get("macroType");
32     if (macroText != null) {
33         macroService.processSnippetMacro(macroText);
34     } else {
35         System.out.println("Invalid request: macroType is missing");
36     }
37 }
```

Рисунок 5.2. Клас MacroController.

Цей контролер, показаний на Рисунках 5.1 та 5.2, обробляє HTTP-запити для роботи з макросами. Контролер забезпечує:

- Обробку макросів форматування через метод `processFormattingMacro()`.
- Обробку макросів вставки сніппетів через метод `processSnippetMacro()`.

MacroController передає текст макросу до сервісу MacroService для подальшої обробки.

5. MacroService

```
1 package com.example.text_editor.service;
2
3 import com.example.text_editor.template_method.FormattingMacroProcessor;
4 import com.example.text_editor.template_method.MacroProcessor;
5 import com.example.text_editor.template_method.SnippetMacroProcessor;
6 import org.springframework.stereotype.Service;
7
8 3 usages new *
9 @Service
10 public class MacroService {
11
12     1 usage new *
13     public void processFormattingMacro(String macroText) {
14         MacroProcessor processor = new FormattingMacroProcessor();
15         processor.processMacro(macroText);
16     }
17
18     1 usage new *
19     public void processSnippetMacro(String macroText) {
20         MacroProcessor processor = new SnippetMacroProcessor();
21         processor.processMacro(macroText);
22     }
23 }
```

Рисунок 6. Клас MacroService.

Цей клас, показаний на Рисунку 5, реалізує бізнес-логіку обробки макросів. MacroService:

- Використовує FormattingMacroProcessor для обробки форматування.
- Використовує SnippetMacroProcessor для обробки сніппетів.
- Інкапсулює логіку виконання макросів і забезпечує можливість розширення функціоналу.

Посилання на код на GitHub: https://github.com/vvvvvvvvvviktoriiia/text_editor

Діаграма класів для патерну «Template method»:

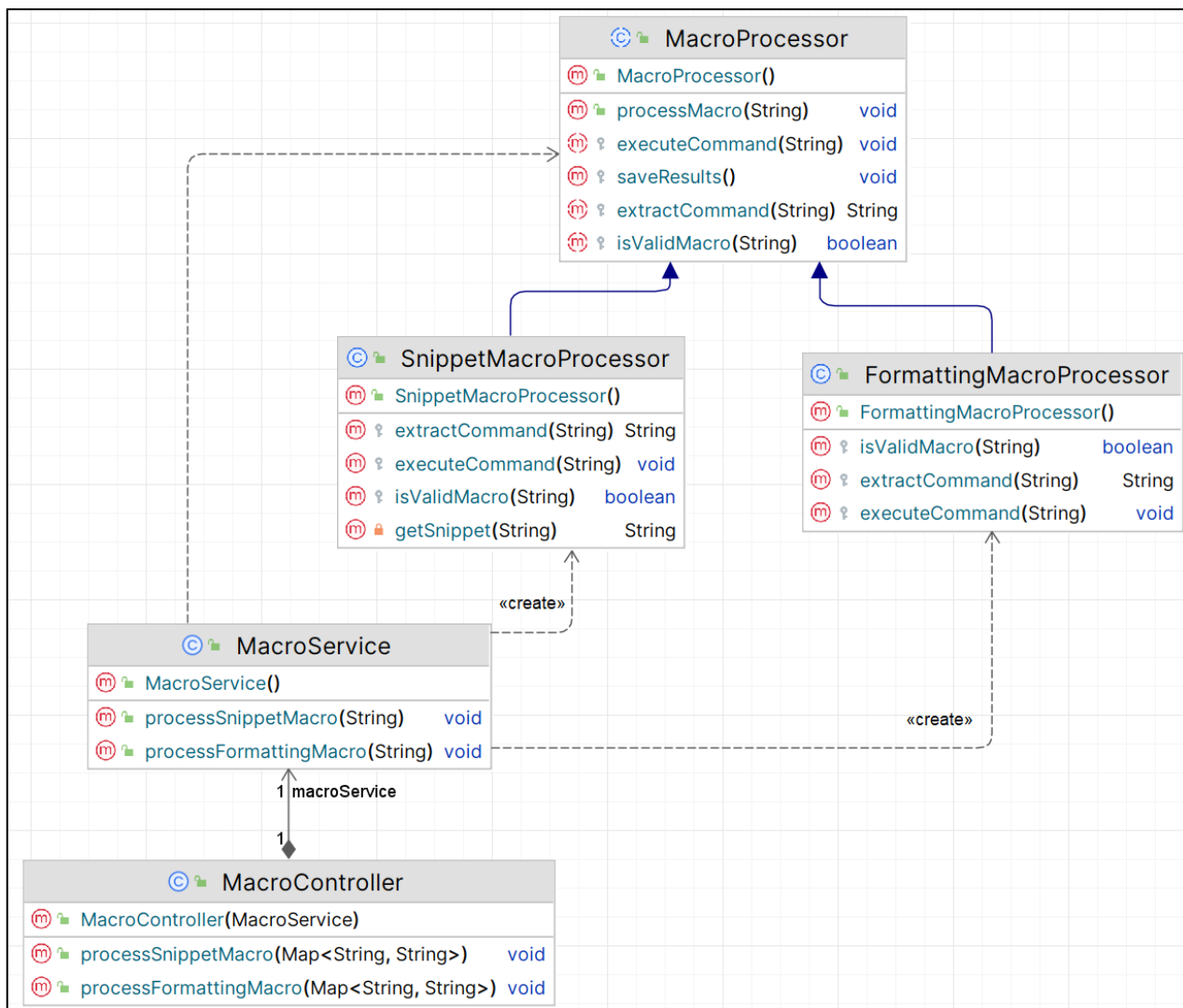


Рисунок 8. Діаграма класів для патерну «Template Method».

Раніше логіка обробки макросів для текстового редактора могла бути реалізована безпосередньо у контролерах або сервісах, що призвело б до:

- Ускладнення підтримки коду: Дублювання алгоритмів для кожного типу макросів у різних місцях.
- Порушення принципу відкритості/закритості: Додавання нового типу макросів вимагало б змін в існуючих методах.
- Відсутності гнучкості: Важко було б забезпечити розширення функціоналу без переробки коду.

Патерн "Template Method" вирішив цю проблему, дозволивши:

- Стандартизацію алгоритму обробки макросів через абстрактний клас **MacroProcessor**, який визначає шаблонний метод `processMacro(String macroText)`.

- Ізоляцію специфічної логіки для кожного типу макросів у відповідних підкласах (FormattingMacroProcessor та SnippetMacroProcessor).
- Легкість розширення: Додавання нових типів макросів без змін у базовому коді.

Переваги застосування патерну «Template method»:

1. Стандартизація:
 - Єдиний алгоритм обробки макросів, що включає перевірку валідності, виконання команди та збереження результатів.
2. Гнучкість:
 - Додавання нових типів макросів (наприклад, макрос для виконання скриптів) можливе через створення нового підкласу без змін у базовій логіці.
3. Принцип відкритості/закритості:
 - Існуючі класи не потрібно змінювати для додавання нових функцій.
4. Модульність:
 - Кожен тип макросу реалізується окремим класом (FormattingMacroProcessor та SnippetMacroProcessor), що полегшує розуміння та тестування.
5. Уніфікація алгоритму:
 - Загальний метод processMacro() виконує однакову послідовність дій для всіх типів макросів, що зменшує дублювання коду.

Висновок: у даній лабораторній роботі реалізовано патерн "Template Method", що дозволило стандартизувати та спростити обробку макросів у текстовому редакторі. Завдяки цьому патерну: логіка обробки макросів стала уніфікованою та легко розширюваною, додавання нових типів макросів більше не вимагає змін у базовій структурі коду, а також система стала модульною, зрозумілою та зручною для підтримки.

Переваги реалізації включають ізоляцію логіки обробки кожного типу макросів, забезпечення єдиної послідовності виконання та підвищення гнучкості системи. Реалізація цього патерну зробила текстовий редактор масштабованим і зручним для подальшого розвитку.